

Mate Helper

Model Creation Guide

How to create your own desktop pet character
from scratch using a template

Table of Contents

1. Introduction
2. Project Structure
3. Quick Start: Using the Template
4. model.py - Identity & Settings
5. phrases.py - Fallback Dialogue
6. strings.py - UI Translations
7. Sprites - Character Artwork
8. Fonts & Ringtone (Optional)
9. Loading Your Model
10. Tips & Best Practices

1. Introduction

Mate Helper lets you create custom desktop pet characters, called 'models'. Each model defines a character's identity, appearance, personality, voice, and dialogue. This guide walks through every file needed to build a model from scratch.

A ready-to-use template is provided at:

```
desktop_pet/models/custom_model/
```

Copy this folder and rename it to create your own character. All template files contain detailed comments explaining each setting.

2. Project Structure

Models live inside the desktop_pet/models/ directory:

```
desktop_pet/models/
  custom_model/      <-- template (copy this!)
  default_models/    <-- built-in characters
    pt/kasane_teto/
    pt/white_guy/
    en/kasane_teto/
    ...
```

Files in a model folder

File	Required	Purpose
__init__.py	Yes	Empty file, marks folder as Python package
model.py	Yes	Identity, AI prompts, sprite names, font, TTS
phrases.py	Yes	Fallback dialogue when no AI is connected
strings.py	Yes	UI translations (pt, en, jp)
sprites/	Yes	Character images (PNG or GIF)
font.ttf	No	Custom font for speech bubbles
ringtone.mp3	No	Custom alarm ringtone

3. Quick Start: Using the Template

Follow these steps to create a new pet character:

Step 1: Copy the template

```
cd mate-helper  
cp -r desktop_pet/models/custom_model desktop_pet/models/my_pet
```

Step 2: Create sprites

Replace the placeholder sprites with your own artwork. Each mood needs a regular and a Speaking variant (9 files total). See Section 7 for requirements.

Step 3: Edit model.py

Change the identity settings:

```
MODEL_ID = "my_pet"  
PET_NAME = "My Pet"  
PET_SHORT_NAME = "Pet"
```

Then customize SYSTEM_PROMPT to define your character's personality.

Step 4: Edit phrases.py

Add your character's dialogue. Used when no AI is connected.

Step 5: Edit strings.py

Customize the UI strings or keep the defaults.

Step 6: Run and select your model

```
python3 desktop_pet/main.py
```

Right-click the pet > Modelo do pet > select your model.

4. model.py - Identity & Settings

This file defines who your character is and how it behaves.

Identity

- MODEL_ID - Short ID matching the folder name (e.g. 'my_pet')
- PET_NAME - Full display name (e.g. 'Kasane Teto')
- PET_SHORT_NAME - Short name for chat (e.g. 'Teto')

Sprites

SPRITE_NAMES maps emotion names to sprite filenames:

```
SPRITE_NAMES = {
    "Normal": "Default",
    "Feliz": "Happy",
    "Triste": "Sad",
    "Raiva": "Angry",
    "Danca": "Dancing",
}
```

AI System Prompt

SYSTEM_PROMPT defines your character's personality:

```
SYSTEM_PROMPT = (
    f"Voce e {PET_NAME}, um gato magico falante.\n"
    "- Responda em portugues brasileiro\n"
    "- Seja sarcastico e engraçado"
)
```

Accessibility Prompts & Tasks

ACCESSIBILITY_SCREEN_PROMPT and ACCESSIBILITY_AUDIO_PROMPT tell the AI how to describe screen captures and audio transcriptions.

ACCESSIBILITY_TASKS configures intervals for automatic tasks. Mode: 'aleatorio' (random) or 'exato' (fixed).

TTS Voices

Voice IDs for each text-to-speech provider:

```
TTS_VOICE = {
    "fish_audio": "",
    "edge_tts": "pt-BR-FranciscaNeural",
    "pyttsx3": "brazil",
}
```

5. phrases.py - Fallback Dialogue

When no AI provider is configured, keyword-matched phrases are used.

Phrase Lists

Each category is a list of strings. The app picks randomly:

```
GREETING = [  
    "Ola! Como voce esta?",  
    "Oieeee!",  
]  
HOW_ARE_YOU = [  
    "Como voce esta?",  
    "Tudo bem?",  
]
```

Keyword Matching

```
CATEGORY_KEYWORDS = {  
    "greeting": ["ola", "oie", "oi"],  
    "thanks": ["obrigado", "valeu"],  
    "bye": ["tchau", "ate logo"],  
}
```

Required Functions

- pick(category, default="") - Returns a random phrase from a category
- get_fallback(message, history) - Matches a message and returns a response

6. strings.py - UI Translations

Provides UI text strings for menus and dialogs:

```
STRINGS = {  
    "pt": {"greeting": "Ola! ^_^", ...},  
    "en": {"greeting": "Hello! ^_^", ...},  
}
```

The template includes all required keys for Portuguese and English.

7. Sprites - Character Artwork

Required Files

Filename	Mood
Default.png	Idle - not speaking
DefaultSpeaking.png	Idle - speaking
Happy.png	Happy - not speaking
HappySpeaking.png	Happy - speaking
Sad.png	Sad - not speaking
SadSpeaking.png	Sad - speaking
Angry.png	Angry - not speaking
AngrySpeaking.png	Angry - speaking
Dancing.png	Dancing (alarm animation)

Missing sprites fall back to Default.png.

Format Recommendations

- PNG: 96x96 pixels (or any consistent size)
- GIF: animated GIFs with frame delays preserved
- Sprite sheets: multiple frames in one PNG, auto-detected
- Use transparent background (alpha channel)

Speaking Animation

Speaking variants (e.g. DefaultSpeaking.png) display when talking. If missing, the non-speaking version is used.

8. Fonts & Ringtone (Optional)

Custom Font (font.ttf)

Place a .ttf file in the model folder for custom speech bubble text. Set FONT_NAME and FONT_SIZE in model.py.

Custom Ringtone (ringtone.mp3)

An MP3 file played during alarms. The Dancing animation plays while the ringtone is active. Set RINGTONE_PATH in model.py, or None to disable alarm audio.

9. Loading Your Model

Once your model folder is complete, the app auto-detects it.

To activate:

- Start the app: `python3 desktop_pet/main.py`
- Right-click the pet > Modelo do pet
- Select your model from the list

History is stored separately per model at `~/.config/teto-pet/history/{MODEL_ID}.json`

Models placed directly under `desktop_pet/models/` work across all languages, ideal for custom characters.

10. Tips & Best Practices

System Prompt

- Be specific about personality traits and speech patterns
- Specify language explicitly
- Add rules like "Never say you are an AI"
- Keep it under 500 characters

Sprites

- All sprites should be the same dimensions
- Use transparency (the window has no background)
- 96x96 pixel art works great

Phrases

- Add at least 5-10 phrases per category
- Use {PET_NAME} and {PET_SHORT_NAME} in phrases
- ALARM_PHRASES show during alarms

Testing

- Use 'Frases prontas' mode first to test phrases
- Then switch to an AI provider to test the prompt
- Check terminal logs if the model doesn't appear
- Ensure __init__.py exists (can be empty)